



WSQ

Learner Guide

For

AI Vibe Coding for iOS Mobile Apps Development

TGS-2023037545

Software Design 3 · ICT-DES-3005-1.1

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN 201200696W

Version 1.0 · 12 September 2026

Document Version Control Record

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	12 September 2026	Initial issue: portable C++ core, Objective-C++ adapter, SwiftUI shell, AI-assisted TDD, ten Activities and aligned assessments.	Tertiary Infotech Academy

Table of Contents

Section	Page
How to Use This Guide · Learning Outcomes · Architecture	3–4
Topic 1 · Requirements and the Portable Component Boundary	5–7
Activities 1–2	8–11
Topic 2 · C++ Methods, Toolchain and AI-Assisted TDD	12–14
Activities 3–4	15–18
Topic 3 · Controls, Features and the SwiftUI Shell	19–21
Activities 5–6	22–25
Topic 4 · Objective-C++ Interoperability and Validation	26–28
Activities 7–8	29–32
Topic 5 · Architecture Evidence and Deployment	33–35
Activities 9–10	36–39
Assessment Preparation and Source Register	40

How to Use This Guide

The regulated C++ outcomes are delivered through a portable domain core. Objective-C++ provides the language-translation adapter, while Swift and SwiftUI provide the native iOS shell. Swift supplements the approved scope; it does not replace K1–K5 or A1–A5.

Detailed step-by-step procedures are contained in this guide and the individual Activity folders. The slide deck remains mechanism-led and does not duplicate these procedures.

Learning Outcomes

- LO1: Determine basic software components using C++ methodologies to meet functional specifications.
- LO2: Apply C++ methodologies and tools for software creation.
- LO3: Select essential C++ controls and features to meet software design requirements.
- LO4: Examine the interoperability and functionality of C++ software components.
- LO5: Generate C++ design documentation aligned with user specifications.

Regulated Criteria

Pair	Knowledge	Ability
K1 / A1	Design requirements for simple, basic software components.	Design a simple software component or interface according to functional specifications and business requirements.
K2 / A2	Basic software design tools and techniques.	Utilise appropriate software design methods and tools, in line with organisational software design practice and principles.
K3 / A3	Types of controls, elements and features in software.	Identify relevant controls, elements and features to meet software design objectives.
K4 / A4	Indicators of software functionality and interoperability.	Assess functionality and interoperability of different elements or components in the software design.
K5 / A5	Documentation of design details.	Produce detailed design documentation mapped to user specifications.

Technical Architecture

Layer	Responsibility	Boundary rule
SwiftUI shell	Controls, navigation, accessibility and state	Sends intent; does not own business rules
Objective-C++ adapter	Type, ownership and error translation	Public header exposes Objective-C-compatible values only

Layer	Responsibility	Boundary rule
Portable C++ core	Invariants, exact values, ports and domain tests	Imports no Apple framework
Evidence pack	Traceability, tests, ADRs and release decisions	Every claim links to reproducible evidence

Topic 1: Requirements and the Portable Component Boundary

Mapping: LO1 · K1 · A1

LO1: Determine basic software components using C++ methodologies to meet functional specifications.

Functional requirements become contracts

Mechanism	Technical design	Evidence
Elicit need → State invariant → Define interface → Write acceptance	<pre>struct Requirement { std::string id; bool testable; };</pre>	Accepted behaviours / specified behaviours

Design fields: actor, input, rule, output

Typical failure: A prompt says build an app but never defines observable success.

Control: Translate each user story into an invariant and testable acceptance criterion.

Retain as evidence: Requirement ID linked to interface and test

Separate business rules from the iOS shell

Mechanism	Technical design	Evidence
SwiftUI event → Adapter call → C++ rule → Value result	<pre>class BudgetPolicy { public: Decision evaluate(const Input&) const; };</pre>	Domain tests runnable without Simulator

Design fields: View, Bridge, Domain, Result

Typical failure: Rules leak into View bodies and cannot be reused or isolated.

Control: Keep portable policy in C++; keep presentation and Apple frameworks in Swift.

Retain as evidence: Dependency diagram plus pure C++ test

Define a narrow C ABI seam

Mechanism	Technical design	Evidence
Choose values → Stabilise types → Expose function → Wrap in Swift	<pre>extern "C" BudgetResult budget_evaluate(BudgetInput input);</pre>	Boundary surface measured in stable operations

Design fields: int64_t, const char*, opaque handle, status code

Typical failure: C++ templates or STL containers cross directly into Swift.

Control: Expose C-compatible data at the boundary; hide implementation behind the adapter.

Retain as evidence: Bridge header with ownership notes

Model money without floating-point drift

Mechanism	Technical design	Evidence
Accept decimal → Convert to cents → Compute in integer → Format for UI	<pre>struct Money { std::int64_t cents; };</pre>	Exact equality for ledger totals

Design fields: minor units, currency, rounding, display

Typical failure: Binary floating point creates a one-cent mismatch.

Control: Represent monetary amounts as signed 64-bit minor units in the domain.

Retain as evidence: C++ unit test with edge amounts

Design immutable value objects

Mechanism	Technical design	Evidence
Validate input → Construct value → Pass by const ref → Return new value	<pre>class Category { std::string value_; public: explicit Category(std::string); };</pre>	Invalid states rejected at construction

Design fields: invariant, constructor, const, equality

Typical failure: Partially initialised objects travel through the system.

Control: Make invariants constructor-level and expose values through read-only accessors.

Retain as evidence: Value-object test matrix

Map a user story to an interface

Mechanism	Technical design	Evidence
Read story → Find verb → Name operation → Specify result	<pre>virtual AddResult addExpense(const ExpenseDraft&) = 0;</pre>	Traceable story-to-operation mapping

Design fields: story, command, query, error

Typical failure: The API mirrors screen widgets instead of business intent.

Control: Name operations after domain decisions, then let SwiftUI adapt them.

Retain as evidence: Interface sketch annotated with requirement IDs

Use acceptance criteria as design pressure

Mechanism	Technical design	Evidence
Given state → When action → Then outcome → And evidence	<pre>// Given valid draft, when added, then total increases by exact cents.</pre>	Criteria with deterministic evidence

Design fields: precondition, trigger, postcondition, artifact

Typical failure: Acceptance criteria describe implementation steps rather than outcomes.

Control: Write externally observable outcomes before selecting controls or tools.

Retain as evidence: Given-When-Then table

Choose component granularity

Mechanism	Technical design	Evidence
Find responsibility → Measure cohesion → Inspect coupling → Cut boundary	ExpenseService -> ExpenseRepository; ViewModel -> ExpenseService;	Reasons-to-change per component

Design fields: responsibility, cohesion, dependency, contract

Typical failure: A single Manager object owns UI, persistence, validation and policy.

Control: Partition by responsibility and dependency direction, not by file-count targets.

Retain as evidence: Responsibility map

Express failure in the contract

Mechanism	Technical design	Evidence
Enumerate faults → Classify recoverability → Create result type → Map to message	using AddResult = std::variant<ExpenseId, DomainError>;	Failure variants handled explicitly

Design fields: validation, storage, bridge, unexpected

Typical failure: A Boolean failure discards the cause and repair path.

Control: Use typed errors in C++; translate them once at the Objective-C++ edge.

Retain as evidence: Error taxonomy and mapping table

Estimate before implementation

Mechanism	Technical design	Evidence
Slice capability → Expose unknowns → Prototype seam → Commit scope	// Spike: Swift -> ObjC++ -> C++ -> immutable result	Unknowns retired before full build

Design fields: must, should, risk, spike

Typical failure: The hardest bridge uncertainty appears late in the capstone.

Control: Time-box an Objective-C++ seam spike before committing the architecture.

Retain as evidence: Spike result and revised scope

Requirements baseline and change control

Mechanism	Technical design	Evidence
Version baseline → Record decision → Link change → Re-run impact	struct Change { std::string requirementId; std::string rationale; };	Changes with explicit downstream impact

Design fields: requirement id, version, rationale, affected tests

Typical failure: A model-generated change silently alters an approved requirement.

Control: Treat AI output as a proposal against a versioned baseline.

Retain as evidence: Change record with affected interfaces and tests

Activity 1: Requirements and Component Boundary

Mapping	Time	Tools
K1, A1	70 minutes	Markdown, diagrams.net or PowerPoint, JSON fixture

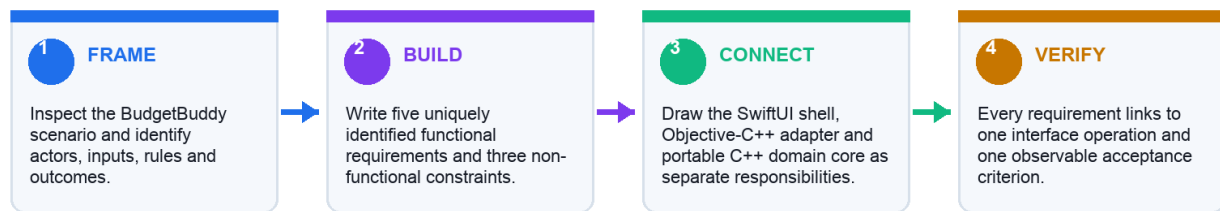
Objective

Convert stakeholder needs into testable requirements and design the SwiftUI–Objective-C++–C++ boundary.

Deliverable

Requirement matrix, boundary diagram and acceptance criteria

Activity 1 workflow · K1, A1



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 1 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

1. Open a terminal at the repository root and run ``cd activities/01-requirements-component-boundary`` followed by ``find . -maxdepth 3 -type f | sort``. Confirm that the scenario, template, solution example and mock user stories are present.
2. Read the brief with ``sed -n '1,160p' starter/scenario.txt`` and inspect the structured stories with ``python3 -m json.tool mock-data/user-stories.json``. Record the two actors and their intended outcomes before proposing components.
3. Copy ``starter/requirements-template.txt`` to a working file named ``requirements-working.txt``. Do not edit the supplied starter or solution reference.
4. Add five unique requirement IDs ``REQ-001`` through ``REQ-005``. For every row, complete actor, input, business rule, observable outcome and acceptance evidence; avoid implementation verbs such as click or instantiate.

5. For the money rules, state the unit explicitly as signed 64-bit integer cents. Add acceptance examples for a valid expense, zero amount, negative amount and an expense above the remaining budget.
6. Create `boundary-working.txt` and write the dependency chain `SwiftUI View -> BudgetViewModel -> BudgetBridge.h -> BudgetBridge.mm -> ExpenseService -> ExpenseRepository`.
7. Under the chain, assign one responsibility to every layer. State that `DomainCore` imports no Swift, Objective-C, UIKit, SwiftUI or Foundation type.
8. For each requirement, name the domain operation that owns the business verb, such as `BudgetPolicy::evaluate`, rather than naming a screen widget.
9. Review the completed matrix against `solution/requirements-matrix.txt` and the boundary against `solution/boundary-diagram.txt`. Differences are acceptable only when they preserve the regulated A1 design intent.
10. Run `rg -n 'REQ-[0-9]{3}|SwiftUI|Objective-C|C|C++' requirements-working.txt boundary-working.txt`. Expect all five IDs and all three technology boundaries to be visible.
11. Create `acceptance-review.txt` and record PASS or REVISE for uniqueness, testability, explicit units, observable outcomes and inward dependency direction.
12. Verification: a reviewer must be able to select any `REQ-\*` row and point to exactly one interface operation plus one reproducible acceptance observation. Retain the three working text files as Activity evidence.

Verification

TEST IT — Every requirement links to one interface operation and one observable acceptance criterion.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

If the boundary follows screen widgets, restate each operation as a business verb before redrawing.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Retain the versioned design artifacts; remove only disposable sketches.

Folder: activities/01-requirements-component-boundary/

Activity 2: Portable C++ Domain Core

Mapping	Time	Tools
K1, A1	85 minutes	C++20, CMake, CTest

Objective

Implement exact-cent value objects and a small budget policy as a standalone CMake library.

Deliverable

Compiling C++20 library and domain tests

Activity 2 workflow · K1, A1



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 2 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

13. Run ``cd activities/02-portable-cpp-domain-core`` and inspect the project with ``find starter -maxdepth 3 -type f | sort``. Confirm the CMake file, public header, implementation and test file exist.
14. Create an isolated working copy with ``cp -R starter workspace``. All edits for this Activity go under ``workspace/``.
15. Open ``workspace/include/BudgetPolicy.hpp``. Confirm ``Money`` stores ``std::int64_t`` cents and that ``BudgetPolicy::evaluate`` accepts limit, spent and draft values without an Apple framework type.
16. Open ``workspace/src/BudgetPolicy.cpp``. Replace the placeholder behaviour with guard clauses that reject a negative limit, negative spent value and a draft amount less than or equal to zero.
17. Add an over-budget guard: reject when ``draft.cents > limit.cents - spent.cents``. For every rejection, return the unchanged remaining amount rather than subtracting the invalid draft.
18. Retain the accepted path as ``limit.cents - spent.cents - draft.cents``. Do not convert to ``double`` and do not introduce a currency-formatted string in the core.
19. Extend ``workspace/tests/BudgetPolicyTests.cpp`` with assertions for valid, zero, negative and over-budget cases. Each branch in the implementation must have at least one named or clearly separated assertion.

20. If CMake is installed, run ``cmake -S workspace -B workspace/build``, then ``cmake --build workspace/build``, then ``ctest --test-dir workspace/build --output-on-failure``. Expect one test target and a 100% pass summary.
21. If CMake is unavailable, run ``mkdir -p workspace/build && c++ -std=c++20 -Wall -Wextra -Werror -I workspace/include workspace/src/BudgetPolicy.cpp workspace/tests/BudgetPolicyTests.cpp -o workspace/build/DomainCoreTests``.
22. Run the fallback binary with ``workspace/build/DomainCoreTests`` and then ``printf '%s\n' $?``. Expect exit status ``0`` and no assertion failure.
23. Run ``rg -n 'UIKit|SwiftUI|Foundation|NSString|NSArray' workspace/include workspace/src``. Expect no matches; any match means the portable boundary has been violated.
24. Compare the working implementation with ``solution/`` only after tests pass. Record the commands and results in ``workspace/verification.txt``, then remove only ``workspace/build/`` when cleanup is required.

Verification

TEST IT — CTest reports all domain tests passed and the public header contains no Objective-C or Swift constructs.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

Delete only the local build directory and reconfigure; inspect the first compiler diagnostic, not the last.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Remove build/; keep source, headers and tests.

Folder: activities/02-portable-cpp-domain-core/

Topic 2: C++ Methods, Toolchain and AI-Assisted TDD

Mapping: LO2 · K2 · A2

LO2: Apply C++ methodologies and tools for software creation.

CMake defines a portable build graph

Mechanism	Technical design	Evidence
Declare target → Set standard → Attach tests → Build out-of-tree	<code>add_library(DomainCore STATIC src/Budget.cpp)</code>	Reproducible build from clean checkout

Design fields: library, C++20, CTest, build dir

Typical failure: Xcode-only configuration hides a broken standalone core.

Control: Make the core buildable with CMake and add it to Xcode as an explicit dependency.

Retain as evidence: Clean configure/build/test transcript

Use Red-Green-Refactor with AI

Mechanism	Technical design	Evidence
Write failing test → Ask for minimal patch → Run test → Refactor with suite	<code>TEST(BudgetPolicy, RejectsNegativeLimit)</code> <code>{ EXPECT_THROW(...); }</code>	Every production change preceded by a meaningful failure

Design fields: red, minimal, green, cleanup

Typical failure: The assistant writes implementation and tests that merely echo it.

Control: Freeze expected behaviour first and demand the smallest causal patch.

Retain as evidence: Failing trace, diff and passing trace

Prompt context is an engineering input

Mechanism	Technical design	Evidence
State contract → Attach relevant files → Set constraints → Define done	Goal: implement <code>evaluate()</code> . Preserve <code>BudgetPolicy.hpp</code> . Run <code>ctest</code> .	Acceptance checks passed on first review

Design fields: goal, context, boundary, checks

Typical failure: A broad prompt invites API churn and unrelated edits.

Control: Name protected interfaces and exact verification commands.

Retain as evidence: Prompt plus reviewed diff

Select types to encode invariants

Mechanism	Technical design	Evidence
Inspect range → Choose signedness → Name unit → Guard conversion	enum class ExpenseKind : std::uint8_t { food, travel, other };	Invalid conversions detected at boundaries

Design fields: int64_t, enum class, chrono, optional

Typical failure: Primitive strings and doubles hide units and optionality.

Control: Prefer domain types, scoped enumerations and explicit optional values.

Retain as evidence: Type table with rejected examples

Functions expose intention

Mechanism	Technical design	Evidence
Name verb → Limit parameters → Return value → Keep pure	[[nodiscard]] Money remaining(Money limit, Money spent);	Pure functions covered by deterministic tests

Design fields: command, query, const, nodiscard

Typical failure: A function mutates globals and communicates failure by logging.

Control: Return data or typed errors; inject side effects through interfaces.

Retain as evidence: Function signature review

Control flow should reveal policy

Mechanism	Technical design	Evidence
Guard invalid → Select case → Apply rule → Return result	if (draft.amount.cents <= 0) return DomainError::invalidAmount;	Cyclomatic paths each have a named test

Design fields: guard clause, switch, loop, result

Typical failure: Deep nesting obscures which condition owns a decision.

Control: Use guards and small policy functions; reserve loops for collections.

Retain as evidence: Branch-to-test matrix

RAII makes ownership visible

Mechanism	Technical design	Evidence
Acquire → Bind to object → Use safely → Release by scope	auto repo = std::make_unique<JsonExpenseRepository>(path);	Zero manual delete paths

Design fields: unique_ptr, vector, fstream, destructor

Typical failure: Raw ownership crosses the bridge and leaks on an error path.

Control: Keep RAII inside C++; cross the boundary using copied value snapshots.

Retain as evidence: Sanitizer run and ownership diagram

Dependency injection enables substitution

Mechanism	Technical design	Evidence
Define port → Inject implementation → Exercise fake → Swap adapter	ExpenseService(ExpenseRepository& repo, Clock& clock);	Core tests run without disk or iOS runtime

Design fields: interface, constructor, fake, production

Typical failure: A singleton couples rules to persistence and time.

Control: Inject repositories and clocks through narrow abstract ports.

Retain as evidence: Fake-backed unit test

Compiler diagnostics are design feedback

Mechanism	Technical design	Evidence
Enable warnings → Treat as errors → Read locus → Repair cause	target_compile_options(DomainCore PRIVATE -Wall -Wextra -Werror)	Zero warnings in clean build

Design fields: -Wall, -Wextra, -Werror, sanitizer

Typical failure: Generated code compiles only under permissive defaults.

Control: Use strict warnings and sanitizers during development; document platform differences.

Retain as evidence: Warning-free build log

Review AI diffs semantically

Mechanism	Technical design	Evidence
Check contract → Trace data → Inspect lifetime → Run regression	// Review: conversion from Swift Int to int32_t may overflow.	Review findings resolved before merge

Design fields: API, invariant, ownership, tests

Typical failure: A plausible patch introduces silent truncation or ownership ambiguity.

Control: Review data representation and lifetimes before style.

Retain as evidence: Annotated diff with disposition

Organisational practice is executable

Mechanism	Technical design	Evidence
Write convention → Automate check → Run locally → Enforce in CI	cmake --build build && ctest --test-dir build --output-on-failure	Same checks locally and in CI

Design fields: format, lint, tests, evidence

Typical failure: A design principle remains prose and is ignored under time pressure.

Control: Translate practices into formatter, compiler and test configuration.

Retain as evidence: Tool configuration committed beside code

Activity 3: AI-Assisted TDD for C++

Mapping	Time	Tools
K2, A2	75 minutes	AI coding assistant, CTest, diff review

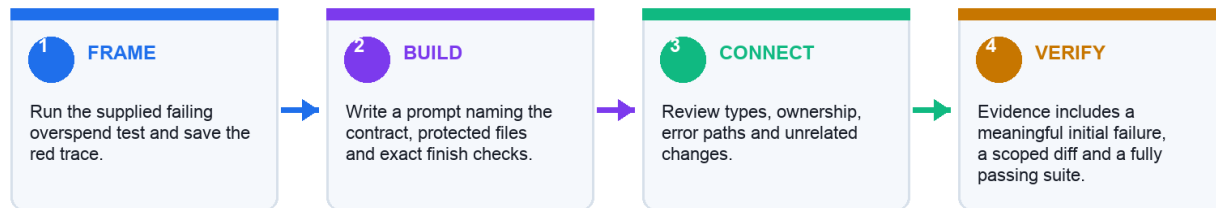
Objective

Use an evidence-bound AI prompt to implement a rule through Red-Green-Refactor.

Deliverable

Prompt contract, failing trace, minimal patch and passing trace

Activity 3 workflow · K2, A2



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 3 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

25. Run ``cd activities/03-ai-assisted-tdd`` and inspect ``starter/failing-test.cpp``, ``starter/prompt-contract.txt`` and the completed examples under ``solution/``.
26. Create ``workspace/``, then copy the header and implementation produced in Activity 2 into it. If that output is unavailable, copy the corresponding files from ``../02-portable-cpp-domain-core/solution/include`` and ``../02-portable-cpp-domain-core/solution/src``.
27. Copy ``starter/failing-test.cpp`` into ``workspace/`` and compile it with ``c++ -std=c++20 -Wall -Wextra -Werror -I workspace workspace/BudgetPolicy.cpp workspace/failing-test.cpp -o workspace/red-test``.
28. Run ``workspace/red-test``; expect a non-zero exit or assertion failure if the overspend rule is missing. Save the terminal output and exit status in ``workspace/red-trace.txt`` before changing production code.
29. Copy ``starter/prompt-contract.txt`` to ``workspace/prompt-contract.txt``. Complete Outcome, Relevant context, Protected interfaces, Constraints, Exact verification commands and Finish condition.

30. Give the completed contract plus only the relevant header, implementation and failing test to the AI coding assistant. Instruct it to propose the smallest patch and not to change the method signature or cents unit.
31. Review the proposal before applying it. Reject changes that introduce floating point, alter unrelated formatting, remove the failing assertion or add an Apple dependency.
32. Apply the accepted lines to `workspace/BudgetPolicy.cpp`. Capture a unified review artifact with `diff -u ../02-portable-cpp-domain-core/solution/src/BudgetPolicy.cpp workspace/BudgetPolicy.cpp > workspace/review.diff || true`.
33. Recompile using the same command, run `workspace/red-test`, and record the now-zero exit status in `workspace/green-trace.txt`.
34. Run the complete Activity 2 regression suite against the changed implementation, not only the new overspend test. Expect valid, zero, negative and over-budget cases to pass.
35. Complete a copy of `solution/review-checklist.txt`. Every unchecked item requires a repair followed by focused and regression reruns.
36. Retain prompt contract, red trace, reviewed diff, green trace and checklist. These five artifacts demonstrate Red-Green-Refactor rather than an unverified AI-generated result.

Verification

TEST IT — Evidence includes a meaningful initial failure, a scoped diff and a fully passing suite.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

If the test passed before the change, strengthen the assertion before asking for implementation.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Keep prompt and evidence; revert experimental changes outside the accepted patch.

Folder: activities/03-ai-assisted-tdd/

Activity 4: Toolchain and Quality Gates

Mapping	Time	Tools
K2, A2	65 minutes	CMake, compiler warnings, sanitizers, shell

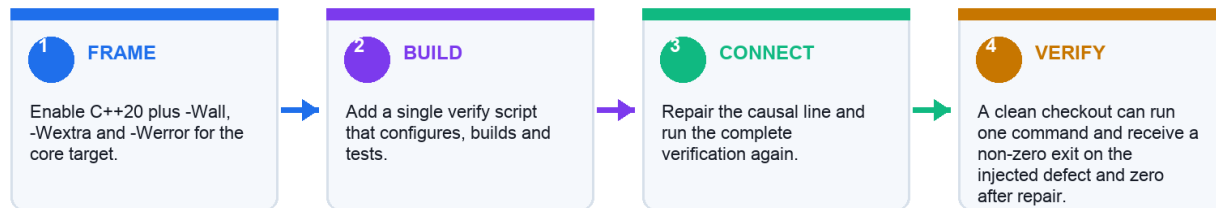
Objective

Turn organisational design practices into compiler, formatter and test gates.

Deliverable

Warning-clean build, quality script and evidence log

Activity 4 workflow · K2, A2



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 4 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

37. Run ``cd activities/04-toolchain-quality-gates`` and create ``workspace/``. Copy the completed Activity 2 project into it with ``cp -R ../02-portable-cpp-domain-core/solution workspace/domain``.
38. Inspect ``workspace/domain/CMakeLists.txt`` and compare it with ``solution/CMakeLists-snippet.txt``. Confirm C++20 is required and warnings are enabled only for supported compiler families.
39. Merge the strict warning block into the working CMake file. Keep ``-Wall -Wextra -Werror``; do not silence a diagnostic globally to make the build pass.
40. Copy ``starter/verify.sh`` to ``workspace/verify.sh`` and run it explicitly with ``sh workspace/verify.sh``. Expect configure, build and CTest to complete with exit status zero when CMake is available.
41. When CMake is unavailable, verify the equivalent gate with ``c++ -std=c++20 -Wall -Wextra -Werror -I workspace/domain/include workspace/domain/src/BudgetPolicy.cpp workspace/domain/tests/BudgetPolicyTests.cpp -o workspace/DomainCoreTests && workspace/DomainCoreTests``.
42. Inject a controlled defect by adding ``int unused = 1;`` inside ``BudgetPolicy::evaluate``. Re-run the strict build and confirm the unused-variable diagnostic makes the command fail.

43. Copy the failing diagnostic and command into ``workspace/evidence-log.txt``; record the source file and line reported by the compiler.
44. Remove the injected variable at its causal line. Do not weaken ``-Werror`` and do not add an unused suppression.
45. Run the complete gate again and record the successful exit status. Compare your evidence log with ``solution/evidence-log.txt`` for the required Red/Repair/Green structure.
46. Optional when supported: configure with ``cmake -S workspace/domain -B workspace/asan-build -DENABLE_ASAN=ON``, build, and run tests. Record whether the platform supports the sanitizer configuration.
47. Run ``find workspace -maxdepth 2 -type f | sort`` and identify generated binaries or cache data separately from source and verification scripts.
48. Cleanup only ``workspace/domain/build``, ``workspace/asan-build`` and compiled binaries. Retain the strict configuration, verification script and evidence log as A2 evidence.

Verification

TEST IT — A clean checkout can run one command and receive a non-zero exit on the injected defect and zero after repair.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

If Apple Clang rejects a flag, gate it by compiler ID and document the equivalent.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Remove generated build and sanitizer output, not the verification script.

Folder: activities/04-toolchain-quality-gates/

Topic 3: Controls, Features and the SwiftUI Shell

Mapping: LO3 · K3 · A3

LO3: Select essential C++ controls and features to meet software design requirements.

SwiftUI renders state as a function

Mechanism	Technical design	Evidence
Own state → Derive view → Send intent → Render update	@Observable final class BudgetViewModel { private(set) var snapshot: Snapshot }	UI states covered by previews or tests

Design fields: @State, @Observable, body, action

Typical failure: Business rules execute inside body and repeat on every render.

Control: Keep View declarative; send intent to a state owner that calls the bridge.

Retain as evidence: State-transition diagram

Select controls from the data contract

Mechanism	Technical design	Evidence
Identify type → Choose affordance → Constrain input → Expose feedback	Picker("Category", selection: \$draft.category) { ... }	Invalid states prevented or explained

Design fields: TextField, Picker, Toggle, Button

Typical failure: A free-text field captures a value from a small controlled set.

Control: Choose the control that communicates range, optionality and consequence.

Retain as evidence: Control-selection matrix

Validation belongs at two boundaries

Mechanism	Technical design	Evidence
Guide in UI → Validate domain → Map error → Preserve input	guard let cents = MoneyParser.cents(text) else { message = .invalidAmount }	Rejected invalid inputs with recoverable state

Design fields: format, invariant, message, recovery

Typical failure: UI validation is treated as sufficient security or correctness.

Control: Provide immediate SwiftUI guidance and enforce invariants again in C++.

Retain as evidence: Negative-path test plus UI message

Navigation models workflow state

Mechanism	Technical design	Evidence
Define destination → Bind path → Pass identity → Handle return	enum Route: Hashable { case expenseDetail(UUID); case settings }	Deep link and back path produce valid state

Design fields: NavigationStack, NavigationPath, route, selection

Typical failure: Views pass mutable models through an implicit navigation chain.

Control: Route with identifiers and reload authoritative snapshots at boundaries.

Retain as evidence: Navigation-state map

Lists need stable identity

Mechanism	Technical design	Evidence
Fetch snapshot → Identify row → Render collection → Apply intent	ForEach(model.snapshot.expenses, id: \.id) { ExpenseRow(\$0) }	Row updates affect exactly one domain entity

Design fields: Identifiable, UUID, ForEach, delete

Typical failure: Array indices are used as identity and shift after deletion.

Control: Expose stable IDs from the core and reconcile snapshots by identity.

Retain as evidence: List mutation test

Swift concurrency guards responsiveness

Mechanism	Technical design	Evidence
Start task → Leave main actor → Await bridge work → Publish state	Task { await model.refresh() }	No blocking work on main thread

Design fields: Task, async, actor, MainActor

Typical failure: Disk or model inference runs synchronously from a button action.

Control: Isolate expensive work and publish UI state on the main actor.

Retain as evidence: Responsiveness trace

Accessibility is part of control design

Mechanism	Technical design	Evidence
Name purpose → Expose value → Support focus → Test scaling	.accessibilityLabel("Delete expense")	Critical flow works with VoiceOver and large text

Design fields: label, value, hint, Dynamic Type

Typical failure: An icon-only control has no accessible name.

Control: Treat semantic labels and adaptable layout as acceptance criteria.

Retain as evidence: Accessibility audit notes

Previews are executable design probes

Mechanism	Technical design	Evidence
Create fixture → Inject fake → Render states → Inspect variants	<pre>#Preview("Error") { BudgetScreen(model: .fixture(.error)) }</pre>	Representative states preview without network or storage

Design fields: empty, loaded, error, large text

Typical failure: A preview depends on production data and never reaches failure states.

Control: Inject deterministic snapshots and display state extremes.

Retain as evidence: Preview gallery

Feature flags bound experimental AI

Mechanism	Technical design	Evidence
Define flag → Default off → Observe usage → Remove or graduate	<pre>if flags.isEnabled(.smartCategory Suggestion) { ... }</pre>	Experimental path has owner and expiry

Design fields: identifier, default, telemetry, owner

Typical failure: An AI-generated feature becomes permanent hidden complexity.

Control: Gate experiments, capture evidence and set a review date.

Retain as evidence: Flag register and decision record

Local Foundation Models stay supplemental

Mechanism	Technical design	Evidence
Check availability → Prepare bounded prompt → Validate output → Fall back	<pre>guard modelSystem.isAvailable else { return .manualEntry }</pre>	Deterministic core works when model is unavailable

Design fields: availability, schema, validation, fallback

Typical failure: A local model becomes a hidden source of business truth.

Control: Use model output as a suggestion; validate before committing to the core.

Retain as evidence: Fallback test and decision trace

Design selections require rationale

Mechanism	Technical design	Evidence
List candidates → Score fit → Prototype risk → Record choice	<pre>// Decision: Picker because category is closed and required.</pre>	Each material control linked to objective

Design fields: control, criterion, trade-off, decision

Typical failure: Controls are selected because they are familiar or visually fashionable.

Control: Record why the chosen control better enforces the requirement.

Retain as evidence: Control rationale table

Activity 5: Objective-C++ Adapter

Mapping	Time	Tools
K3, A3	85 minutes	Xcode, Objective-C++, XCTest

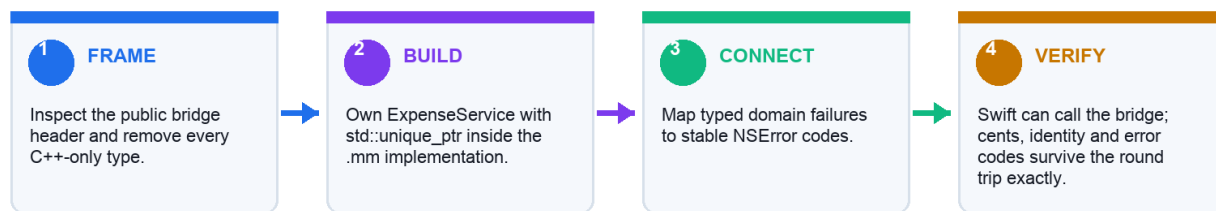
Objective

Expose the portable domain core to Swift through a narrow Objective-C-compatible API.

Deliverable

Bridge header, .mm adapter and round-trip test

Activity 5 workflow · K3, A3



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 5 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

49. Run ``cd activities/05-objective-cpp-adapter`` and inspect the starter API with ``sed -n '1,200p' starter/BudgetBridge.h`` and ``sed -n '1,200p' starter/BudgetBridge.mm``.
50. Create a working copy with ``cp -R starter workspace``. Place the Activity 2 ``BudgetPolicy.hpp`` on the Objective-C++ target's header search path; do not import it from ``BudgetBridge.h``.
51. In the Xcode app target, add ``workspace/BudgetBridge.h`` and ``workspace/BudgetBridge.mm``. Confirm the implementation suffix is ``.mm``; a ``.m`` file cannot compile C++ constructs.
52. Keep the public header limited to Foundation-compatible declarations, fixed-width integers, Objective-C objects and ``NSError **``. Run ``rg -n 'std::|#include <vector>|#include <string>' workspace/BudgetBridge.h`` and expect no matches.
53. In ``BudgetBridge.mm``, include ``BudgetPolicy.hpp`` and add a private ``std::unique_ptr<BudgetPolicy>`` owner. Initialise it once in ``-init``.
54. Translate ``limit``, ``spent`` and ``draft`` from ``int64_t`` into ``Money`` value objects, call ``evaluate``, then copy accepted and remaining-cents values into ``BudgetResultDTO``.
55. Map a rejected domain decision to the stable error domain ``BudgetDomain`` and code ``1001``. Do not expose a raw C++ exception or pointer to Swift.

56. Add the supplied ``solution/bridge-test.swift`` pattern to an XCTest target. Ensure the app module is imported with ``@testable import`` when appropriate.
57. Run the focused test in Xcode, or use ``xcodebuild test -scheme BudgetBuddy -destination 'platform=iOS Simulator,name=iPhone 17 Pro' -only-testing:BudgetBuddyTests/BudgetBridgeTests`` after substituting the actual scheme and available simulator.
58. Expect the exact input ``10_000``, spent ``2_500`` and draft ``1_250`` to return remaining cents ``6_250``; add a second test that verifies rejection maps to error code ``1001``.
59. If Swift cannot see the API, check target membership, the bridging header or module import, and that no C++ type appears in the public header before changing the domain core.
60. Retain the public header, ``.mm`` adapter, passing test result and a short ownership note showing Swift retains the adapter while the adapter owns the C++ service through RAI.

Verification

TEST IT — Swift can call the bridge; cents, identity and error codes survive the round trip exactly. Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

If Swift cannot import the header, check target membership and confirm C++ includes appear only in ``.mm``.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Clean derived build data only if needed; retain tests and mapping table.

Folder: `activities/05-objective-cpp-adapter/`

Activity 6: SwiftUI Controls and State

Mapping	Time	Tools
K3, A3	80 minutes	SwiftUI, Observation, Xcode previews

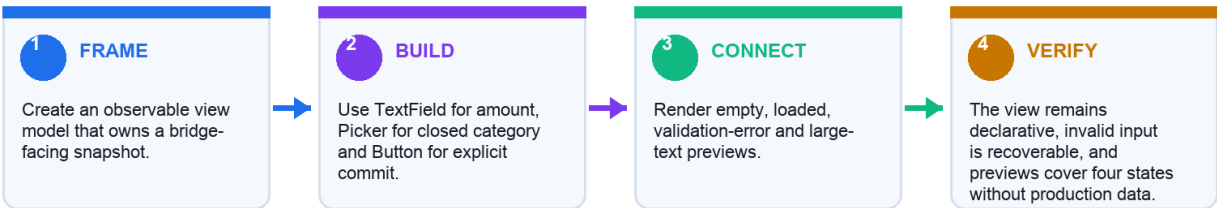
Objective

Build an accessible SwiftUI expense-entry shell whose controls reflect the domain contract.

Deliverable

SwiftUI entry flow, deterministic previews and validation states

Activity 6 workflow · K3, A3



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 6 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

61. Run ``cd activities/06-swiftui-controls-state`` and inspect both starter Swift files. Create a working copy with ``cp -R starter workspace``.
62. Add ``workspace/BudgetViewModel.swift`` and ``workspace/BudgetScreen.swift`` to the app target that already contains the Activity 5 bridge.
63. In ``BudgetViewModel``, keep draft text and category as editable state, but expose the accepted budget snapshot with ``private(set)`` so Views cannot bypass the bridge.
64. Implement a parser that converts the decimal amount string into integer cents. Reject empty, non-numeric, zero and negative input without clearing the learner's draft.
65. Implement ``save()`` to call ``BudgetBridge``. Map the stable bridge error into a user-facing message and update the snapshot only when the domain result is accepted.
66. In ``BudgetScreen``, add a ``TextField`` for formatted amount entry, a ``Picker`` for the closed category set and a ``Button`` for the explicit save consequence. Do not place budget policy in ``body``.
67. Add ``accessibilityIdentifier("amount")``, ``accessibilityIdentifier("saveExpense")`` and a semantic label for any icon-only control.

68. Create deterministic preview fixtures for empty, loaded and validation-error states. Add one preview with a large Dynamic Type environment.
69. Build the app and use the preview or Simulator to enter `12.50`, select a category and save. Expect the bridge to receive `1250` cents and the accepted snapshot to refresh once.
70. Enter `0`, `-1` and non-numeric text. Expect a clear validation message, no domain mutation and the original draft to remain available for correction.
71. Turn on VoiceOver or use Accessibility Inspector. Navigate the amount, category and save controls in logical order and confirm each announces purpose and current value.
72. Retain the two Swift files, four preview states, exact-cent observation, negative-path observation and accessibility note as the Activity evidence.

Verification

TEST IT — The view remains declarative, invalid input is recoverable, and previews cover four states without production data.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

If previews fail, inject the fixture model and remove filesystem work from initialisation.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Retain preview fixtures; remove screenshots containing personal data.

Folder: activities/06-swiftui-controls-state/

Topic 4: Objective-C++ Interoperability, Persistence and Validation

Mapping: LO4 · K4 · A4

LO4: Examine the interoperability and functionality of C++ software components.

Objective-C++ owns language translation

Mechanism	Technical design	Evidence
Receive Swift value → Convert to C++ → Call domain → Return Objective-C value	// BudgetBridge.mm auto result = core_ >add(toCpp(draft));	One translation point per operation

Design fields: .h, .mm, namespace, NSError

Typical failure: C++ types leak through the public Objective-C header.

Control: Keep the header Objective-C compatible and put C++ includes in .mm.

Retain as evidence: Bridge compile test

Ownership across ARC and RAIL

Mechanism	Technical design	Evidence
Swift retains bridge → Bridge owns unique_ptr → Core owns values → Scope releases	std::unique_ptr<ExpenseService> service_;	No leaks or dangling pointers under repeated use

Design fields: strong, unique_ptr, value, deinit

Typical failure: Swift stores a pointer to a temporary C++ object.

Control: Let the adapter own long-lived C++ services; return copied immutable DTOs.

Retain as evidence: Leak-check run

Map typed errors once

Mechanism	Technical design	Evidence
Receive variant → Extract error → Assign code → Create NSError	NSError *error = [NSError errorWithDomain:BudgetDomain code:code userInfo:info];	Stable error codes reach Swift tests

Design fields: domain code, domain, message, recovery

Typical failure: Every screen invents a different string for the same domain failure.

Control: Centralise DomainError-to-NSError mapping in the adapter.

Retain as evidence: Error mapping test table

DTOs freeze the boundary shape

Mechanism	Technical design	Evidence
Read domain → Copy fields → Create DTO → Consume in Swift	@interface ExpenseDTO : NSObject @property(readonly) int64_t cents; @end	Swift sees immutable, complete records

Design fields: identity, amount, category, timestamp

Typical failure: Swift receives multiple arrays whose indices must stay aligned.

Control: Use explicit DTO objects with stable identity and documented units.

Retain as evidence: DTO contract test

JSON persistence stays behind a port

Mechanism	Technical design	Evidence
Serialise snapshot → Write atomically → Read data → Validate schema	repository.save(snapshot); // port hides JSON and filesystem	Restart reproduces last accepted state

Design fields: encoder, temp file, rename, schemaVersion

Typical failure: A partial write corrupts the only saved file.

Control: Write to a temporary file, fsync where relevant, then replace atomically.

Retain as evidence: Restart and corruption tests

Schema evolution needs a version

Mechanism	Technical design	Evidence
Read version → Select migration → Transform → Persist current	if (doc.schemaVersion == 1) return migrateV1(doc);	Legacy fixtures load into current model

Design fields: v1, migration, default, current

Typical failure: A new field makes existing user data unreadable.

Control: Version persisted documents and test every supported migration path.

Retain as evidence: Fixture-based migration suite

Interoperability has observable indicators

Mechanism	Technical design	Evidence
Build both targets → Call boundary → Compare values → Inspect failures	EXPECT_EQ(swiftRoundTrip.amountCents, cppInput.amount.cents);	Contract tests pass for success and failure paths

Design fields: compile, round-trip, identity, error

Typical failure: Only a happy-path simulator demo is used as evidence.

Control: Measure compile compatibility, value fidelity, error fidelity and lifecycle safety.

Retain as evidence: Interop scorecard

Contract tests isolate the seam

Mechanism	Technical design	Evidence
Arrange adapter → Send DTO → Invoke core → Assert mapped result	<code>func testNegativeAmountMapsToSta bleErrorCode() throws { ... }</code>	Adapter regression detected without UI automation

Design fields: fixture, input, call, output

Typical failure: Bridge tests are postponed until full end-to-end testing.

Control: Test the translation seam directly with deterministic core doubles.

Retain as evidence: Bridge contract suite

End-to-end tests prove user value

Mechanism	Technical design	Evidence
Launch fixture → Enter expense → Save → Relaunch and verify	<code>app.buttons["saveExpense"].tap (); XCTAssertTrue(app.staticTexts["\$12.50"].exists)</code>	Critical journey passes from control to persisted core state

Design fields: launch arg, accessibility id, persistence, assertion

Typical failure: A UI test checks labels but not the C++ decision or persistence result.

Control: Choose a thin golden path that crosses every architectural boundary.

Retain as evidence: XCUITest trace and screenshot

Failure injection tests recovery

Mechanism	Technical design	Evidence
Inject fault → Observe mapping → Show recovery → Retry safely	<code>repo.failNextSave = true // deterministic test double</code>	Failure states preserve user input and offer a valid next action

Design fields: disk full, invalid JSON, bridge error, retry

Typical failure: Errors are logged but the UI remains stuck in a loading state.

Control: Inject controlled faults and assert state transitions plus recovery action.

Retain as evidence: Negative-path evidence matrix

Functionality needs a scorecard

Mechanism	Technical design	Evidence
Choose criteria → Set threshold → Run suite → Review evidence	<code>ReleaseGate{.domain=1.0, .brid ge=1.0, .ui=0.95, .crashes=0};</code>	All release gates meet declared thresholds

Design fields: correctness, latency, accessibility, recovery

Typical failure: A build is declared ready because it compiles.

Control: Combine domain, bridge, UI and quality indicators into one release decision.

Retain as evidence: Release scorecard with links

Activity 7: Persistence and Error Recovery

Mapping	Time	Tools
K4, A4	75 minutes	C++ filesystem, JSON fixture, CTest

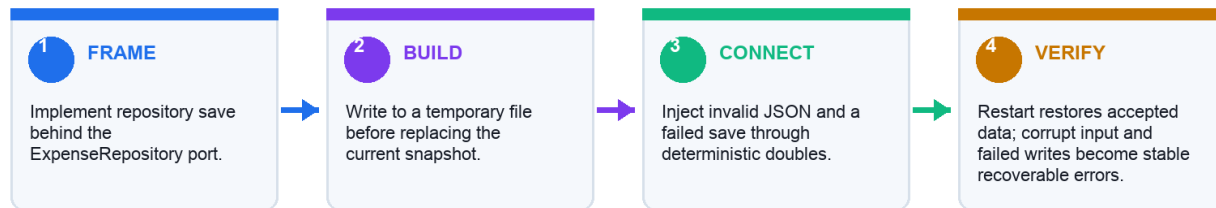
Objective

Persist domain snapshots safely and prove recovery from invalid data and write failure.

Deliverable

Versioned JSON repository, migration fixture and recovery tests

Activity 7 workflow · K4, A4



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 7 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

73. Run ``cd activities/07-persistence-error-recovery`` and validate the supplied fixture with ``python3 -m json.tool starter/snapshot-v1.json``. Expect ``schemaVersion`` 1 and amount values expressed as integer cents.
74. Inspect ``starter/repository-contract.hpp``. Confirm the domain depends on the abstract ``ExpenseRepository`` port rather than a filesystem path or Foundation API.
75. Create ``workspace/``, copy the repository contract and fixture, and add ``JsonExpenseRepository.cpp`` behind the port used by the domain service.
76. Implement save as a two-file sequence: serialise the complete snapshot to ``<path>.tmp``, verify the write succeeded, then replace the current snapshot. Never truncate the current file before a complete replacement exists.
77. Add ``schemaVersion`` to every saved document. Keep amounts as integer cents and identities as stable strings.
78. Implement the v1 migration shown in ``solution/migration.cpp``: load ``limit``, translate each expense, apply current defaults and return a current-schema snapshot.

79. Write a restart test that saves two accepted expenses, constructs a new repository instance and asserts the same IDs and cent values are restored.
80. Write a corruption test using invalid JSON. Expect a stable ``corruptStore`` result rather than a crash or an empty-success snapshot.
81. Write a failed-save test with a deterministic repository double. Confirm the previously valid snapshot remains readable and the UI-facing draft is not discarded.
82. Validate every JSON fixture with ``python3 -m json.tool <file>`` and keep test paths under a temporary Activity directory, never the real home or application-support directory.
83. Record the restart, migration, corruption and failed-write results in ``workspace/recovery-evidence.txt``. Compare the expected recovery sequence with ``solution/recovery-test.txt``.
84. Delete only temporary test stores and ``.tmp`` files after evidence capture. Retain both schema fixtures and every migration/recovery test.

Verification

TEST IT — Restart restores accepted data; corrupt input and failed writes become stable recoverable errors.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

If tests touch the real home directory, inject a temporary path and fail the test on escape.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Delete only temporary test stores; retain migration fixtures.

Folder: activities/07-persistence-error-recovery/

Activity 8: Interoperability Validation Scorecard

Mapping	Time	Tools
K4, A4	80 minutes	CTest, XCTest, XCUITest, Instruments

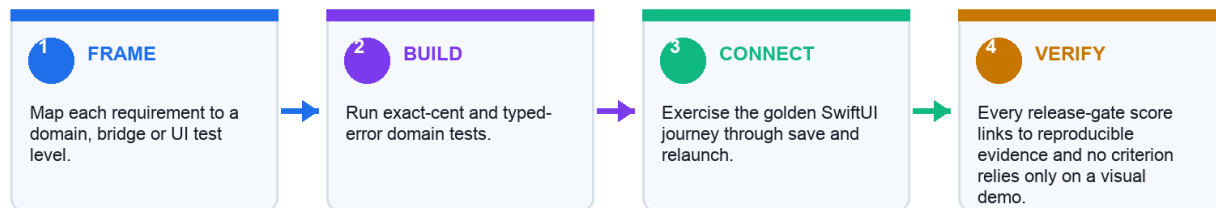
Objective

Assess the C++/Objective-C++/SwiftUI system with domain, bridge and end-to-end evidence.

Deliverable

Test matrix, execution evidence and release scorecard

Activity 8 workflow · K4, A4



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 8 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

85. Run ``cd activities/08-interop-validation-scorecard`` and copy ``starter/test-matrix.csv`` plus ``starter/scorecard.json`` into ``workspace/``.
86. Open the requirement matrix from Activity 1 and assign each requirement to the lowest useful test layer: C++ domain, Objective-C++ bridge, Swift state or end-to-end UI.
87. Complete the domain-test column with exact-cent, invalid-input, overspend and persistence behaviours. Point each entry to an actual command output or test report.
88. Complete the bridge-test column with value fidelity, stable identity, error-code mapping and ownership/lifecycle evidence from Activity 5.
89. Complete the UI-test column with the accepted expense flow, validation recovery, relaunch and accessibility checks from Activities 6–7.
90. Run the portable suite using the Activity 2 CMake command or its strict ``c++`` fallback and save the result under ``workspace/logs/domain.txt``.

91. Run focused XCTest bridge tests and the golden UI journey. Save text results or exported test summaries under ``workspace/logs/bridge.txt`` and ``workspace/logs/ui.txt``.
92. For the golden journey, launch with deterministic fixture data, add ``12.50``, save, relaunch and assert the same amount and identity appear. Capture one screenshot only after the assertion passes.
93. Calculate correctness and interoperability as passed required checks divided by total required checks. Accessibility is a critical-path decision, not a count of labels.
94. Fill ``workspace/scorecard.json`` with numeric scores and a decision of ``hold`` or ``release-candidate``. Do not choose release-candidate when any required criterion lacks evidence.
95. Compare structure with the supplied solution files, then run ``python3 -m json.tool workspace/scorecard.json`` and ``python3 -c 'import csv; list(csv.DictReader(open("workspace/test-matrix.csv")))'``. Expect both commands to exit zero.
96. Retain the matrix, scorecard, raw logs and golden-journey screenshot. Each score must link to evidence rather than confidence or a visual demo alone.

Verification

TEST IT — Every release-gate score links to reproducible evidence and no criterion relies only on a visual demo.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

If a UI failure is ambiguous, reproduce it one layer lower before changing the architecture.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Remove disposable run artifacts; retain the scorecard and final evidence links.

Folder: `activities/08-interop-validation-scorecard/`

Topic 5: Architecture Evidence, Documentation and Deployment

Mapping: LO5 · K5 · A5

LO5: Generate C++ design documentation aligned with user specifications.

C4 views explain different questions

Mechanism	Technical design	Evidence
Context → Container → Component → Code	Context -> iOS App -> SwiftUI Objective-C++ C++ Core	Stakeholder question answered at correct zoom

Design fields: people, deployables, responsibilities, interfaces

Typical failure: A single dense diagram mixes business actors with classes.

Control: Use layered diagrams and state the audience and decision for each.

Retain as evidence: C4 set linked from design index

ADRs preserve design reasoning

Mechanism	Technical design	Evidence
Name decision → Record context → Compare options → State consequences	# ADR-002: Use Objective-C++ as the sole C++ translation boundary	Material choices have retrievable rationale

Design fields: status, context, decision, consequence

Typical failure: Documentation states what exists but not why alternatives were rejected.

Control: Write a short ADR for boundaries, persistence and AI usage.

Retain as evidence: ADR index with requirement links

Traceability closes the specification loop

Mechanism	Technical design	Evidence
Assign IDs → Link design → Link test → Report status	REQ-004 BudgetPolicy BudgetPolicyTests/RejectsNegative PASS	Requirements with design and passing evidence

Design fields: REQ, component, test, evidence

Typical failure: A feature is marked done without proving the approved requirement.

Control: Maintain a bidirectional matrix from requirement to implementation and test.

Retain as evidence: Traceability matrix

API documentation states contracts

Mechanism	Technical design	Evidence
Describe purpose → Name parameters → Define errors → Give example	/// Evaluates a draft. Amount is cents; returns typed error; no mutation.	Consumers implement without reading source

Design fields: precondition, unit, ownership, error

Typical failure: Generated comments repeat the method name and omit units or ownership.

Control: Document invariants, units, lifetime and failure semantics.

Retain as evidence: Header documentation review

Architecture diagrams must match code

Mechanism	Technical design	Evidence
Generate inventory → Compare dependencies → Update diagram → Review diff	SwiftUI -> BudgetBridge -> DomainCore; no reverse dependency	Zero undocumented or impossible dependency arrows

Design fields: target, module, edge, owner

Typical failure: A polished diagram describes an architecture the build no longer uses.

Control: Validate diagrams against target membership and imports before release.

Retain as evidence: Diagram-to-build reconciliation

Documentation is a tested deliverable

Mechanism	Technical design	Evidence
Build docs → Check links → Run examples → Review audience	cmake -S DomainCore -B build && cmake --build build && ctest --test-dir build	New developer reaches a passing test from clean checkout

Design fields: README, links, commands, roles

Typical failure: Setup steps depend on an unstated local path or secret.

Control: Exercise documentation in a clean environment and capture the transcript.

Retain as evidence: Docs validation log

Deployment separates signing from design

Mechanism	Technical design	Evidence
Choose identity → Create archive → Validate build → Prepare release	xcodebuild -scheme BudgetBridgeApp -configuration Release archive	Validated archive with reproducible design evidence

Design fields: bundle id, certificate, archive, metadata

Typical failure: A local simulator run is described as App Store readiness.

Control: Treat signing, archive validation and store submission as separate evidence gates.

Retain as evidence: Archive validation record

Privacy manifests document data use

Mechanism	Technical design	Evidence
Inventory APIs → Classify data → Declare reasons → Verify package	PrivacyInfo.xcprivacy // reviewed against linked frameworks	Declared use matches code and third-party packages

Design fields: API, data, purpose, manifest

Typical failure: An AI-added SDK expands collection without documentation review.

Control: Re-run privacy inventory whenever dependencies or telemetry change.

Retain as evidence: Privacy manifest review

AI review needs accountable evidence

Mechanism	Technical design	Evidence
Request review → Verify claim → Classify finding → Resolve or accept	Finding: cents overflow. Evidence: boundary test with Int.max. Owner: developer.	Accepted findings have owner and rationale

Design fields: prompt, evidence, severity, owner

Typical failure: AI review comments are copied into approval without reproduction.

Control: Require code locus and a repeatable test for material findings.

Retain as evidence: Review register with dispositions

Release notes map value to change

Mechanism	Technical design	Evidence
Name capability → State user impact → List risk → Link evidence	Added exact-cent budgeting; validated restart persistence and VoiceOver flow.	Every release claim points to a checked artifact

Design fields: feature, impact, known issue, verification

Typical failure: Release notes list commits but not observable user outcomes.

Control: Write outcome-oriented notes and disclose known limitations.

Retain as evidence: Release note linked to tests and ADRs

The evidence pack supports handover

Mechanism	Technical design	Evidence
Index artifacts → Record versions → Assign owners → Set review date	EvidencePack v1.0: requirements, ADRs, tests, validation, release checklist	A reviewer can reproduce the release decision

Design fields: design, tests, deployment, operations

Typical failure: Evidence is scattered across local screenshots and chat history.

Control: Package requirements, design, tests, risks and deployment evidence with provenance.

Retain as evidence: Signed evidence index

Activity 9: Architecture and Traceability Documentation

Mapping	Time	Tools
K5, A5	75 minutes	Markdown, Mermaid or editable shapes, Doxygen-style comments

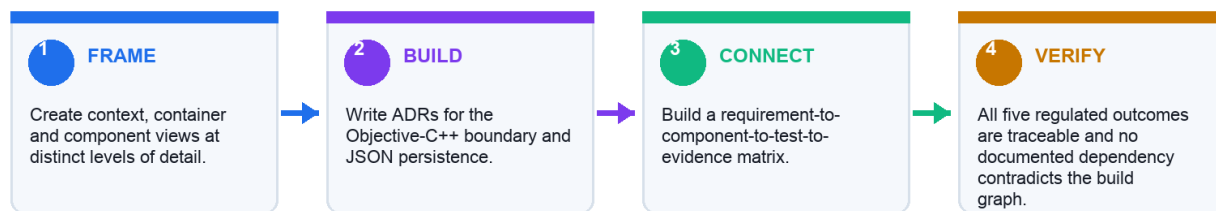
Objective

Create design documentation that maps user specifications to architecture, interfaces and tests.

Deliverable

C4 diagrams, two ADRs, API contract and traceability matrix

Activity 9 workflow · K5, A5



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 9 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

97. Run ``cd activities/09-architecture-traceability-docs`` and create ``workspace/docs``. Copy ``starter/adr-template.txt`` and ``starter/traceability.csv`` into that folder.
98. Create a context view showing the budget owner, BudgetBuddy iOS app and local storage. Keep source editable in a diagram tool or PowerPoint; export a review copy only after labels fit.
99. Create a container/component view with SwiftUI shell, BudgetViewModel, BudgetBridge, ExpenseService, BudgetPolicy and ExpenseRepository. Draw arrows in the actual dependency direction.
100. Reconcile the diagram against target membership and imports. Run ``rg -n '^import|^#import|^#include' <your-source-folders>`` and remove any diagram edge not supported by code.
101. Copy the ADR template to ``workspace/docs/ADR-001-objective-cpp-boundary.txt``. Complete status, context, options, decision, consequences, affected requirements, evidence, owner and review date.

102. Create ``ADR-002-json-persistence.txt`` and compare direct Swift persistence, C++ repository port and mixed ownership. Record why the selected port preserves the regulated component boundary.
103. Add API contract comments to the bridge header. Document cents units, nullability, ownership, error domain/codes, preconditions and whether an operation mutates state.
104. Fill ``workspace/docs/traceability.csv`` with one row for each ``REQ-001`` through ``REQ-005``. Each row must name a design element, implementation path, test, evidence path and status.
105. Validate the CSV with ``python3 -c 'import csv; rows=list(csv.DictReader(open("workspace/docs/traceability.csv"))); assert len(rows)==5; assert all(r["status"] for r in rows)``. Expect five populated rows and exit status zero.
106. Compare the ADR and traceability structure with the solution examples. Do not copy a decision that conflicts with your actual code or evidence.
107. Ask a peer to choose two requirements at random and follow the matrix in both directions—from requirement to evidence and from test back to requirement.
108. Retain editable diagram sources, exported review images, two ADRs, documented bridge API and validated traceability CSV. Remove only obsolete exported drafts.

Verification

TEST IT — All five regulated outcomes are traceable and no documented dependency contradicts the build graph.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

If the diagram is crowded, split by stakeholder question instead of shrinking labels.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Retain source diagrams and Markdown; remove obsolete exported drafts.

Folder: `activities/09-architecture-traceability-docs/`

Activity 10: Capstone: BudgetBuddy Release Evidence

Mapping	Time	Tools
A1, A2, A3, A4, A5	105 minutes	C++20, CMake, Xcode, SwiftUI, XCTest, documentation templates

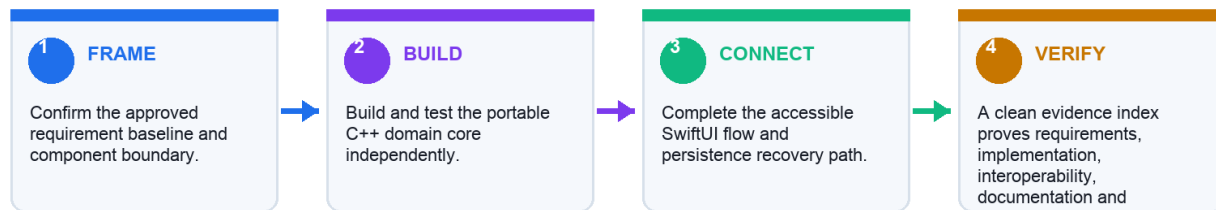
Objective

Integrate the portable C++ core, Objective-C++ adapter and SwiftUI shell into a verified release evidence pack.

Deliverable

Working app, complete tests, architecture pack and deployment checklist

Activity 10 workflow · A1, A2, A3, A4, A5



Detailed commands and expected observations follow below. Retain the named evidence before continuing.

Figure: Activity 10 workflow from framing through retained verification evidence.

Preparation

- Open the matching starter and mock-data files.
- Confirm the workspace contains no credentials or personal information.
- Record the baseline or failing test.
- If using AI, state protected files and exact verification commands.

Detailed Procedure

109. Run ``cd activities/10-capstone-release-evidence``, read ``starter/capstone-brief.txt``, and copy the checklist plus evidence index into ``workspace/``.
110. Create ``workspace/app``, ``workspace/tests``, ``workspace/docs`` and ``workspace/evidence``. Copy only the accepted outputs from Activities 1–9; exclude build caches, credentials, signing files and personal data.
111. Reconfirm the requirement baseline by comparing the Activity 1 matrix with the capstone brief. Record any change as an explicit decision before changing implementation.
112. Build and test the portable C++ core independently using ``cmake -S workspace/app/DomainCore -B workspace/app/DomainCore/build && cmake --build workspace/app/DomainCore/build && ctest --test-dir workspace/app/DomainCore/build --output-on-failure``, or the documented strict compiler fallback.

113. Build the Xcode scheme and run focused bridge tests. Record the scheme, destination and result; do not describe a Simulator build as an archive or App Store submission.
114. Run the SwiftUI golden journey: add an exact-cent expense, trigger and recover from an invalid entry, save, relaunch and verify stable identity and amount.
115. Run the accessibility critical path and persistence fault injection. Place raw results under ``workspace/evidence/`` and link them from the scorecard.
116. Reconcile C4 diagrams, ADRs and the public bridge contract against the final code. Resolve any dependency or ownership mismatch before marking documentation complete.
117. Complete ``workspace/evidence-index.csv`` with artifact version, requirement IDs, owner, evidence path and status. No PASS row may have an empty evidence field.
118. Review ``starter/release-checklist.txt``. Mark the archive item as planned unless a signed Release archive was actually created and validated; never infer App Store readiness from local tests.
119. Write ``workspace/docs/deployment-position.txt`` using the bounded pattern in the solution: distinguish local release-candidate evidence from signing, upload, processing and submission gates.
120. Run ``rg -n -i 'api[_]?key|secret|password|BEGIN .*PRIVATE KEY' workspace || true``. Investigate every match and remove real secrets before packaging.
121. Validate that every requirement has design, implementation, test and evidence links. Ask a reviewer to reproduce one domain check and one bridge/UI check from the index.
122. Create the final inventory with ``find workspace -type f | sort > workspace/evidence/file-inventory.txt``. Retain sources, fixtures, tests and evidence; remove only caches, derived data and temporary stores.

Verification

TEST IT — A clean evidence index proves requirements, implementation, interoperability, documentation and deployment readiness without claiming App Store submission.

Retain the exact command output, test result, matrix or screenshot named by the deliverable.

Troubleshooting

When a system test fails, localise the fault by rerunning domain then bridge tests before editing UI code.

Localise failures from C++ domain to Objective-C++ bridge, Swift state and finally the visible interface.

Cleanup

Remove local build caches and secrets; retain source, fixtures, tests and evidence pack.

Folder: `activities/10-capstone-release-evidence/`

Assessment Preparation

Written Assessment: five open-ended questions covering K1–K5, one hour. Practical Performance: three tasks covering A1–A5, one hour.

Submit candidate work through <https://lms-tms.tertiaryinfotech.com/>. Answer keys remain trainer-only.

Source Register

Governed by SOURCE_REGISTER.md, the approved C++ TSC mapping and course proposal, both legacy Swift/iOS decks, the AI-driven Swift architecture reference and the original assessment papers in [reference/tms/](#).